A BEGINNER'S GUIDE

audit_repo.sh
explained line by line

What the script does, how it behaves on your computer, Claude, ChatGPT and GitHub Copilot, and the exact prompts to use. No programming knowledge needed.

1. The 60-second picture

Imagine you are buying a second-hand house. You would not just walk through it once. You would hire a **building inspector** who brings a team of specialists: one checks the locks, one checks the wiring, one checks the plumbing, one hunts for hidden keys taped under the doormat. At the end, the inspector collects every specialist's notes, removes the duplicates, puts the scariest problems first, and hands you one tidy report.

audit_repo.sh is that inspector, for computer code. You give it a project (the 'house'). It runs several free security tools (the 'specialists'), merges what they find, ranks it from most to least dangerous, explains how to fix each problem, and writes it all into a PDF.

The four specialists, in everyday terms

Specialist	What it checks (the house analogy)	Real name
The ingredient checker	Every outside library the project uses is like a bought-in part. Are any of them on a public recall list?	OSV-Scanner, npm audit, Trivy
The key hunter	Did someone leave a password or secret key taped under the doormat (inside the code, or in old versions of it)?	Gitleaks (and Trivy)
The code reader	Reads the project's own code looking for dangerous habits, like leaving a window latch broken.	Semgrep
The settings checker	Looks at set-up files for unsafe choices, like a door that is set to stay unlocked.	Trivy

What the severity words mean

Word	Everyday meaning	Example from the house
Critical	Fix first. Someone could take over or steal everything, easily.	The front door key is published on the internet.
High	Serious. Dangerous on its own or with one more weakness.	A back window that opens with a butter knife.
Medium	Weakness that needs the right conditions, or missing protection.	No alarm system sticker, no fence.
Low	Minor. Fix when convenient.	A squeaky hinge.

> **Honest limits.** The results are produced by tools, not by a human expert. Some findings will be **false alarms** (the tool is nervous, but the 'problem' is harmless). And the tools can only find *known* kinds of problems. Always have a person or an AI assistant double-check the important ones before acting.

2. Words you will meet (mini glossary)

Word	Plain meaning
Script	A text file full of instructions that the computer follows from top to bottom, like a recipe.
Terminal / command line	A window where you type instructions instead of clicking. Where scripts are started.
Repository (repo)	A project's folder, stored online (e.g. on GitHub), including its full history of changes.
Git, commit	Git is the system that remembers every version of a project. A 'commit' is one saved snapshot with an ID.
Clone	Download a complete copy of a repo, history included.
Library / dependency	Ready-made code written by others that a project borrows (like buying a lock instead of forging one).
Lockfile	The exact shopping list of libraries and precise versions a project uses. Scanners need it.
Vulnerability	A weakness an attacker could use. Public ones get catalogue numbers (CVE-2021-12345).
Secret	A password, access key or private key. It must never sit in code, because anyone with the code gets it.
Redact	Black out. The script hides secret values so they never appear in your report.
Scanner	A program that automatically inspects code for known kinds of problems.
JSON	A neat, standard way for programs to store data in a text file, so other programs can read it.
False positive	A false alarm: the scanner flags something that is actually fine.
Sandbox / container	A sealed, temporary computer that an AI chat uses to run code. Often with limited internet.
Exit code	A number a program leaves behind when it finishes: 0 usually means 'went fine'.

3. How to read the line-by-line sections

The script is 491 lines long. The next sections walk through it in order. **Left column:** the line numbers in the file. **Middle column:** the real code (long lines are shortened with ...). **Right column:** what it means in plain English. Lines that are only blank, or are just a closing bracket, are skipped or grouped with their neighbours.
The file has two parts, like a book with two chapters written in two languages:

Part	Lines	Language	Job
A. The conductor	1-172 and 489-491	Bash (the language of the terminal)	Gets the project, runs each tool, writes the attendance sheet.
B. The report writer	174-487	Python (tucked inside the file)	Reads all tool results, merges and ranks them, writes the Markdown and PDF reports.

> **Two helpful ideas.** (1) A **variable** is a labelled box that holds a value, such as TARGET holding the web address you typed. (2) A **function** is a named mini-recipe you can call again and again, such as **have** (is this tool installed?).

Part A. The conductor (Bash, lines 1-172)

A1. Opening notes and ground rules (lines 1-39)

Lines	The code (shortened)	What it means, in plain English
1	`#!/usr/bin/env bash`	Like the label on a game cartridge: it tells the computer which 'console' (a program called Bash) should run this file.
2-25	`# ====== audit_repo.sh ... # USAGE ... # NOTE: ...`	Any line starting with # is a **comment**: a note for humans that the computer ignores. This block is the instruction leaflet: how to start it, the optional switches, which tools it can use, which files you get, and a reminder that results may include false alarms.
6-11	`# ./audit_repo.sh <target> [output-dir]`	How you start it: the script's name, then the **target** (what to inspect), then optionally a results folder. The target can be a GitHub web address, a folder on your computer, or one single file.
13-15	`RUN_BUILD=1 / REPORT_ONLY=1 / NO_PDF=1`	Three optional on/off switches. **RUN_BUILD**: also build the project and run its own tests (riskier, because it runs the project's code). **REPORT_ONLY**: skip scanning and just rebuild the report from earlier results. **NO_PDF**: skip the PDF.
26	`set -u`	A safety rule: if the script ever uses a label nobody filled in, stop instead of guessing. Like a cashier who refuses to ring up an item with no price tag.
28	`usage() { sed -n '2,24p' "$0"; exit 1; }`	Defines a helper named **usage**. When called, it prints lines 2-24 of the script itself (the leaflet above) and stops. ($0 means 'this file'.)
29	`TARGET="${1:-}"; [[ -z "$TARGET" ]] && usage`	Stores the first thing you typed after the script's name in a box called TARGET. If the box is empty (you typed nothing), show the leaflet and stop.
30	`OUT="${2:-./audit-report}"`	Stores the second thing you typed as the results folder. If you typed nothing, it uses a folder called audit-report.
31	`mkdir -p "$OUT"; OUT="$(cd "$OUT" && pwd)"`	Creates the results folder if it does not exist, then swaps its short name for its full address (like replacing 'the shop around the corner' with a street address) so the script never gets lost.
32	`STATUS="$OUT/tool_status.tsv"`	Chooses the name of the **attendance sheet**: a file where every tool will record whether it ran, was skipped, or failed.
34	`have() { command -v "$1" >/dev/null 2>&1; }`	Helper that answers 'is this tool installed?' with yes or no. The >/dev/null part just hides the chatter.
35	`st() { printf ... >> $STATUS; echo ...; }`	Helper (st = status) that writes one line on the attendance sheet (tool, result, note) **and** prints it on screen, like '[failed] trivy - reason'.
36	`one_line() { tr '\n\t' ' ' | cut -c1-220; }`	Squashes a long multi-line error message into one short line (maximum 220 characters) so it fits in a table.
37	`valid_json() { python3 -c "import json..." "$1" ...; }`	A health check for result files: asks Python to open the file as JSON. If it opens, the file is healthy; if not, it is broken or empty.
39	`if ! have python3; then echo " python3 is required"; exit 1; fi`	Python is the one mandatory tool, because it writes the report. If it is missing, say so and stop.

A2. Working out what you gave it (lines 41-48)

Lines	The code (shortened)	What it means, in plain English
42	`MODE=""`	Creates an empty box called MODE. It will end up holding one word: **repo**, **dir** or **file**.
43	`if [[ "${REPORT_ONLY:-0}" != "1" ]]; then`	Only work out the mode in a normal run. (With REPORT_ONLY nothing is scanned, so there is nothing to decide.)
44	`if [[ -f "$TARGET" ]]; then MODE=file`	If the target is a single file on your computer: mode = **file** (the 'snippet' mode).
45	`elif [[ "$TARGET" =~ ^https?:// || ... ^git@ ]]`	Otherwise, if it starts with http:// or https:// (a web address) or git@ (another style of Git address): mode = **repo**.
46	`elif [[ -d "$TARGET" ]]; then MODE=dir`	Otherwise, if it is a folder on your computer: mode = **dir**.
47	`else echo "Target not found: $TARGET"; exit 1; fi`	If it is none of those, say so and stop. Usually a typo in the address or path.

A3. The scanning chapter, called run_scans (lines 50-170)

Everything between line 50 and line 170 is one big named mini-recipe. It is only *defined* here and is actually *performed* on line 172.

Lines	The code (shortened)	What it means, in plain English
50	`run_scans() {`	Starts the named chapter **run_scans**.
51	`: > "$STATUS"`	Wipes the attendance sheet clean so old results never mix with new ones.
52	`local WORK IS_GIT=0 COMMIT="n/a"`	Three labelled boxes used only inside this chapter: WORK (the folder to inspect), IS_GIT (0 = not known to be a Git project yet) and COMMIT (version ID, 'n/a' for now).
53-54	`case "$MODE" in file) ... cp "$TARGET" "$WORK/"`	**If mode is file:** make a fresh folder called snippet inside the results folder and put a **copy** of your file in it. Your original is never touched.
55-58	`repo) WORK="$OUT/repo" ... git clone "$TARGET" "$WORK"`	**If mode is repo:** work in a folder called repo inside the results folder. If it was downloaded before, reuse it (saves time). Otherwise **git clone** it, with its *full history*, because the key hunter needs old versions too. If the download fails, stop.
59	`dir) WORK="$(cd "$TARGET" && pwd)" ;;`	**If mode is dir:** work directly inside your folder (using its full address).
61	`cd "$WORK" || exit 1`	Walk into the folder to be inspected. If that is impossible, stop.
62	`git rev-parse --is-inside-work-tree ... && IS_GIT=1`	Ask: 'is this folder tracked by Git?' If yes, set IS_GIT to 1.
63	`[[ $IS_GIT -eq 1 ]] && COMMIT=$(git rev-parse --short HEAD)`	If it is a Git project, read the short ID of its latest saved version (the commit). It goes on the report so you know exactly which version was checked.
65-66	`{ echo "target=..."; echo "mode=..."; ... } > "$OUT/meta.env"`	Writes a small **label file** recording what was inspected, in which mode, which version, and today's date. The report writer reads it later.

Step 1: Look around (lines 68-86)

Lines	The code (shortened)	What it means, in plain English
69-70	`echo "[*] Overview" if [[ "$MODE" != "file" ]]`	Prints a progress message. For a repo or folder, build an overview. (A single file has little to summarise.)
72-74	`echo "Files tracked: ... Contributors: ..." echo "Last commit: ..."`	For a Git project: counts the files, counts how many people contributed, and shows the date of the last change. A project nobody has touched for years often has outdated, unpatched parts.
75-76	`echo Manifests:; git ls-files | grep -E '(package.json|requirements..|Dockerfile ...)' | head -15`	Lists up to 15 'shopping-list' files such as package.json or requirements.txt (they name the borrowed libraries) and packaging files such as Dockerfile. Line 76's long pattern is just the list of file names to look for.
77-78	`git log --name-only | sort | uniq -c | sort -rn | head -8`	Finds the 8 files changed most often in the project's history. Files edited constantly are **hotspots**, which is where bugs tend to hide.
79-82	`else echo "Not a git repository..." ; find ... | wc -l`	If it is not a Git project: say that history-based checks are skipped and just count the files (ignoring the huge node_modules library folder).
83	`} > "$OUT/overview.txt"`	Everything printed in this block is saved into overview.txt for the report.
84-85	`else echo "Single file: ..." > "$OUT/overview.txt"`	For a single file, the overview is just its name and how many lines it has.

Step 2: Make temporary ingredient lists (lines 88-100)

A **lockfile** is the exact shopping list of libraries and versions. Scanners need it. Many projects do not publish theirs (OWASP Juice Shop deliberately hides its own), so the script writes a temporary one and deletes it afterwards.

Lines	The code (shortened)	What it means, in plain English
89	`GENERATED_LOCKS=()`	An empty notepad, to remember any temporary lockfile we create so we can delete it later.
90	`if [[ "$MODE" != "file" ]] && have npm; then`	Only for repos or folders, and only if npm (the tool for Node.js projects) is installed.
91-92	`while ... read -r pj; do d="$(dirname "$pj")"`	Loop over every package.json found; for each, note the folder it lives in.
93	`if [[ ! -f "$d/package-lock.json" && ! -f ... yarn.lock && ! -f ... pnpm-lock.yaml ]]`	Check whether that folder already has a lockfile of any kind. Only carry on if it has none.
94	`npm install --package-lock-only --ignore-scripts --no-audit --no-fund`	Ask npm to write **only** the shopping list. It downloads no programs, and thanks to --ignore-scripts it runs **none** of the project's own code. The other two flags just silence extras.
95	`GENERATED_LOCKS+=("$d/package-lock.json")`	Write it on the notepad so it can be deleted at the end.
98	`done < <(find . -name package.json -not -path '*/node_modules/*' ... | head -10)`	The list being looped over: every package.json, skipping library and .git folders, at most 10.
99	`[[ ${#GENERATED_LOCKS[@]} -gt 0 ]] && echo "[*] Generated ..."`	If any temporary lockfiles were made, tell you how many.

Step 3: The ingredient checkers (lines 102-125)

Lines	The code (shortened)	What it means, in plain English
104-106	`if [[ "$MODE" == "file" ]]; then st " osv-scanner" "n/a" ...`	A single file has no ingredient list, so both ingredient checkers are marked **n/a** (not applicable) on the attendance sheet.
108-110	`if have osv-scanner; then osv-scanner scan source --no-ignore -r . ...`	**OSV-Scanner** (made by Google) compares every library against a huge public recall list. It is run twice: once saving a computer-readable file (osv.json), once a human-readable one (osv.txt). **--no-ignore** = also look at files the project asked Git to ignore (that is where lockfiles hide). **-r** = search sub-folders too.
111-113	`if grep -qi "No package sources found" ... st ... "failed" ...`	**Judging the result.** If OSV found no ingredient lists, or could not reach its online recall list, record 'failed ... result is NOT clean'. This stops a silent failure from looking like good news. Otherwise record 'ok'.
114	`else st "osv-scanner" "skipped" " not installed ..."`	If OSV is not installed: record 'skipped' with a link to install it.
116-122	`if have npm; then ... npm audit --json > "$OUT/npm-audit$tag.json" . ..`	**npm audit** is npm's own built-in check against its list of known weaknesses. For each lockfile (up to 10) it runs the check and saves the result under a name based on the folder. Line 121 counts how many produced a healthy result.
123-124	`if [[ $n -gt 0 ]]; then st ... "ok" . .. else st ... "n/a" ...`	At least one healthy result = 'ok'. None = 'n/a' (no Node.js lockfile). If npm is not installed: 'skipped'.

Step 4: Trivy, the three-in-one inspector (lines 127-135)

Lines	The code (shortened)	What it means, in plain English
129-131	`trivy fs . --scanners vuln,secret,misconfig --severity CRITICAL,HIGH,MEDIUM,LOW --format json --output ...`	**Trivy** checks three things at once: (1) **vuln** = libraries with known weaknesses, (2) **secret** = passwords or keys left in files, (3) **misconfig** = unsafe settings. It reports all four severity levels and saves the result as JSON, plus a log.
132-134	`if grep -q "FATAL" ... || ! valid_json ...; then st ... " failed" ...; rm -f ...`	If the log contains the word FATAL, or the result file is unreadable, record 'failed' with the first FATAL message, and delete the broken file. Otherwise 'ok'.
135	`else st "trivy" "skipped" ...`	Not installed: skipped.

Step 5: The key hunter, Gitleaks (lines 137-146)

Lines	The code (shortened)	What it means, in plain English
140	`GL=(detect --source . --report-format json --report-path . .. --redact --no-banner)`	**Gitleaks** hunts for secrets left in code. This line prepares its settings: scan this folder; write the result as JSON; and **--redact** = hide the actual secret values, so they never end up in your report.
141	`[[ $IS_GIT -eq 0 ]] && GL+=(--no-git)`	If it is not a Git project, add the setting 'scan the files only; there is no history'.
142	`gitleaks "${GL[@]}" > gitleaks.log 2>&1; rc=$?`	Run it with those settings and remember its exit code (rc), like a report-card number.
143-144	`if [[ $rc -le 1 ]] && valid_json . ..; then st "gitleaks" "ok" ...`	Gitleaks exits with 0 (found nothing) or 1 (found something). **Both are normal**, so success = code 0 or 1 plus a healthy result file. The note says whether Git history was included.
145-146	`else st ... "failed" ... else ... " skipped" ...`	Anything else = failed, with the last log line. Not installed = skipped.

Step 6: The code reader, Semgrep (lines 148-157)

Lines	The code (shortened)	What it means, in plain English
151	`# '--config auto' forces metrics ON, so explicit rule packs are used instead.`	A note to humans, explaining why the simpler setting '--config auto' was avoided: it forces usage tracking on.
152-154	`semgrep scan --config p/security-audit --config p/owasp-top-ten --metrics=off --exclude node_modules --exclude test --exclude '*.min.js' --json -o . ..`	**Semgrep** reads the code itself, hunting for dangerous patterns (for example, building a database command out of text a stranger typed). It uses two rule collections: a general security audit, and the OWASP Top 10 (the best-known list of common web flaws). It turns off usage tracking, skips library, test and minified folders, and saves JSON.
155-156	`if valid_json "$OUT/semgrep.json"; then st ... "ok" else st ... " failed" "needs internet..." ; rm -f . ..`	Healthy file: ok. Otherwise 'failed: needs internet for rule packs' (Semgrep must download its rulebook online), and the broken file is deleted.
157	`else st "semgrep" "skipped" "not installed (pip install semgrep)"`	Not installed: skipped, with the install command.

Step 7: Build and test, optional (lines 159-170)

Lines	The code (shortened)	What it means, in plain English
161	`if [[ "${RUN_BUILD:-0}" == "1" && "$MODE" != "file" ]]; then`	Only if you switched RUN_BUILD on, and it is not a single file.
162-165	`npm install --ignore-scripts ... ; timeout 600 npm test ... ; st ... exit codes`	If it is a Node.js project: install its libraries (still without running their install scripts), then run its tests with a 10-minute limit (timeout 600 = 600 seconds). The results (0 = success) go on the attendance sheet. This is the **only** step that runs the project's own code, which is why it is off by default.
166-167	`else st "build/test" "n/a" ... else st "build/test" "skipped" ...`	Not a Node.js project: n/a. Switch is off: skipped, with the reason.
169	`for f in "${GENERATED_LOCKS[@]:-}"; do ... rm -f "$f"; done`	**Tidy up.** Delete every temporary lockfile we created, leaving the project exactly as we found it.
170	`}`	Closes the run_scans chapter.
172	`if [[ "${REPORT_ONLY:-0}" != "1" ]]; then run_scans; fi`	**Now actually perform the scanning chapter**, unless you asked for REPORT_ONLY.

Part B. The report writer (Python, lines 174-487)

Big idea. Each specialist writes its notes in its own format and its own vocabulary. Part B is like a **translator and editor**: it reads all the notes, translates them into one language, removes duplicates, sorts them, adds advice, and publishes the report.

B1. Hand-over and toolbox (lines 174-212)

Lines	The code (shortened)	What it means, in plain English
174-178	`echo [*] Building report NO_PDF=... python3 - \"$OUT\" <<'__PYREPORT__'`	Prints a progress message, then starts **Python**, handing it the results folder and the NO_PDF switch. Everything from line 179 to the marker on line 487 is a second program tucked inside this file, like a letter inside an envelope. The word __PYREPORT__ is the 'end of letter' signature.
179-182	`import sys, os, json, re, glob, collections OUT = sys.argv[1] ORDER = ["Critical","High","Medium","Low"]`	Fetches Python's toolboxes (for reading files, searching text, counting), receives the results folder, and defines the severity ladder.
184-186	`def load(p): try: return json.load(open(p)) except Exception: return None`	Helper: open a result file and read it. If anything goes wrong, quietly return 'nothing' instead of crashing, so one bad file cannot ruin the whole report.
188-194	`def readkv(p): ...`	Helper: reads simple 'name=value' lines (like meta.env) into an address book, so values can be looked up by name.
196-203	`META = readkv(...) TOOLS = [] ... OVERVIEW = ...`	Reads the three small files made earlier: the label (meta.env), the attendance sheet (tool_status.tsv, split into tool / result / note) and the overview text.
205-207	`def nsev(s): return {...'moderate': 'Medium', 'error': 'High', ...}`	**The translator.** Tools use different words for seriousness ('moderate', 'error', 'warning', 'info'). This converts them all to our four words. Anything unrecognised becomes Medium, the cautious middle.
208-211	`def score_sev(x): ... 'Critical' if x >= 9 else 'High' if x >= 7 ...`	Some tools give a number from 0 to 10 instead of a word. 9 or more = Critical, 7 or more = High, 4 or more = Medium, otherwise Low.
212	`def worse(a, b): ...`	Given two severities, returns the more serious one. Used when two tools disagree about the same problem.

B2. The advice book (lines 214-242)

Lines	The code (shortened)	What it means, in plain English
214-236	`FIXES = [ (r"sql.?inject|sqli...", "Use parameterized queries..."), (r"md5|sha1...", ...), ... ]`	**The advice book.** Each entry pairs a pattern of keywords (like 'sql inject') with plain advice ('use parameterized queries'). There are entries for database injection, command injection, scripts injected into web pages, path tricks, weak password storage, exposed secrets, open CORS, and more.
237	`DEFAULT_FIX = "Review the finding, apply the vendor/advisory remediation..."`	The fallback advice when nothing in the book matches.
238-242	`def fix_for(*texts): ... for rx, msg in FIXES: if re.search(rx, t): return msg`	Collects the words of a finding, lower-cases them, reads down the book, and returns the advice of the **first** match. **Order matters:** the 'weak hash' entry sits above the 'secrets' entry, because a finding named 'md5 used as password' needs hashing advice, not 'rotate the key'. (A real bug found and fixed during testing.)

B3. Collecting everything in baskets (lines 244-260)

Lines	The code (shortened)	What it means, in plain English
245-247	`F = [] DEPS = {} NOTES = []`	Three baskets. **F**: findings about code, secrets and settings. **DEPS**: one entry per library, because several tools may report the same library. **NOTES**: problems reading a result file.
249-250	`def add(kind, sev, src, title, loc, fix): F.append(...)`	Puts one finding into basket F, trimming over-long text so tables stay tidy.
252-260	`def dep(name, sev, src, title, ids, fix, ...): d = DEPS.setdefault(...)`	**The library merger.** When a tool reports a library, this either creates its entry or *adds to* the existing one: it keeps the **worst** severity, remembers every tool that reported it, and gathers descriptions and fix ideas without repeats. So a library flagged by three tools becomes **one row**, not three.

B4. Reading each specialist's notes (lines 262-355)

Lines	The code (shortened)	What it means, in plain English
262-285	`# npm audit ... for f in glob.glob(...): ... fix = ...`	**npm audit's notes.** For each vulnerable library, picks a title and writes the fix. Normal case: 'upgrade to version X'. Special case (important): sometimes every newer version is *also* affected, or npm suggests going back to an *older* version. Then the script says 'No patched release' and suggests replacing the library or pinning a safe version, instead of repeating misleading advice. Any reading problem becomes a note, not a crash.
286-303	`# OSV-Scanner ... sev = nsev(lab) if lab else score_sev(...)`	**OSV's notes.** For each library and weakness: severity from its label, or from its 0-10 score; finds the 'fixed in version' numbers and writes 'Upgrade [library] to [version]'. Hands it to the library merger.
304-321	`# Trivy ... Results ... Vulnerabilities / Secrets / Misconfigurations`	**Trivy's notes** contain three kinds of entries: vulnerable libraries (to the merger), secrets (added as 'Secrets') and unsafe settings (added as 'Config', using Trivy's own recommended fix). A secret's value is never copied.
322-337	`# Gitleaks ... noise = re.compile(...) groups = OrderedDict() ...`	**Gitleaks' notes.** (1) Hits inside test, example, docs or fixture folders are probably fake, so they are merged into **one** Low line. (2) Real hits are grouped by file and kind, so a key committed in 4 old versions shows as one line plus '+3 more in git history'. (3) Severity: private key = **Critical**; 'generic-api-key' = Medium (it often raises false alarms); anything else = High.
338-355	`# Semgrep ... seen = set() ... fix = fix_for(cid, ...)`	**Semgrep's notes.** Skips exact duplicates; builds a readable title from the rule name and adds its CWE tag (an international weakness label, e.g. CWE-89 = database injection); uses Semgrep's own suggested patch if it gave one, otherwise looks the advice up in the advice book.

B5. Merging, sorting, numbering (lines 356-375)

Lines	The code (shortened)	What it means, in plain English
357-365	`ITEMS = list(F) for name, d in DEPS.items(): ... ITEMS.append(dict( kind='Dependencies', ...))`	**Merging.** Takes basket F, then turns each library record into a row: a title, a location like 'lodash 4.17.20 (direct)', and up to two fix suggestions, with the library's own upgrade listed first.
366-367	`KIND_ORDER = {...} ITEMS.sort(key=... )`	Sorts the whole list: most serious first; within one level: Secrets, then Code, Config, Dependencies; then alphabetical by location.
368-370	`cnt = Counter() ... i["id"] = f"{i[\'sev\'][0]}-{cnt[...]:02d}"`	Gives every finding a short ID: the first letter of its severity plus a number, like **C-01** or **H-07**.
371	`json.dump(ITEMS, open(".../findings. json", "w"), indent=2)`	Saves the complete ranked list as **findings.json**, handy for other programs or AI assistants to read.
372-375	`matrix = ... TOTAL = ... failed = [...] skipped = [...]`	Counts findings per category and severity for the summary table, and notes which tools failed or were skipped (this drives the warning banner).

B6. Writing the Markdown report (lines 377-394)

Lines	The code (shortened)	What it means, in plain English
378-392	`md = ["# Security audit report", . ..] for s in ORDER: ... md.append("| ...")`	Builds **REPORT.md**, a plain-text version of the report: header, summary table, tool status, then one table per severity. Written *before* the PDF, so you get something even if the PDF cannot be made.
393-394	`open(".../REPORT.md", " w").write(...) print(...)`	Saves the file and prints a one-line summary on screen, like '53 findings: Critical=11, High=21 ...'.

B7. Drawing the PDF (lines 396-486)

Lines	The code (shortened)	What it means, in plain English
397	`if os.environ.get("NO_PDF") == "1": sys.exit(0)`	If you asked for no PDF, finish here.
398-405	`try: from reportlab... except ImportError: print('PDF skipped: reportlab not installed')`	**reportlab** is the add-on that draws PDFs. If it is not installed, the script politely says so and finishes. You still keep REPORT.md and findings.json. No crash.
407-416	`SEV = {...colors...} body = S(...) def esc(t): ...`	Sets the look: a colour per severity (red, orange, gold, grey), fonts and sizes. **esc()** makes text safe for the PDF by replacing characters the font cannot draw.
417-425	`PDF = ... def foot(c, doc): ... doc = BaseDocTemplate(...) st = []`	Chooses the PDF's file name, defines the footer on every page (target, date, page number), sets paper size and margins, and creates an empty list **st**: the stack of pieces to place on the pages, top to bottom.
427-431	`band = Table([[Paragraph('SECURITY AUDIT REPORT' ...`	Draws the dark-blue title banner: the report name, the target and the details line.
432-435	`warn = "Automated results. ... not manually verified ..."`	Adds the yellow warning box. If any tool failed or was skipped, it adds the bold sentence that the counts are a lower bound and 'not evidence of safety'.
436-447	`Paragraph("1. Summary") ... rows = [hdr] + [...]`	Builds the **summary table**: rows = Secrets, Code, Config, Dependencies and Total; columns = Critical, High, Medium, Low, with big coloured numbers.
448-454	`Paragraph("2. Tool status") ... sc = {"ok": ..., "failed": ...}`	Builds the **Tool status table** from the attendance sheet: green 'ok', red 'failed', gold 'skipped', grey 'n/a', plus the notes. Any reading problems are printed in red underneath.
455-460	`if OVERVIEW.strip(): ... KeepTogether([ovh, ov])`	Adds the grey 'Overview' box (first 26 lines). KeepTogether stops its heading from being left alone at the bottom of a page.
461-476	`Paragraph("3. Findings ...") CAP = 80 for s in ORDER: ...`	The main part: one table per severity with columns **ID, Source, Finding, Location, How to fix**. Capped at 80 rows per severity so the PDF stays readable; the rest stay in findings.json and REPORT.md.
477-485	`steps = [...] doc.build(st)`	Adds the closing 'Recommended order of work' list (secrets first, then injection flaws, then libraries...). **doc.build(st)** then places every piece on pages and writes the PDF file.
486-487	`print(f" PDF written: {PDF}") __PYREPORT__`	Tells you where the PDF is. The marker __PYREPORT__ is the 'end of letter': Python's part is over and Bash takes over again.

Back in Bash: the finish (lines 489-491)

Lines	The code (shortened)	What it means, in plain English
489-491	`echo ; echo "[+] Done. Output directory: $OUT" ; ls "$OUT" | grep -E "^(Security_Audit_Report.pdf|REPORT.md|findings.json)$"`	Prints 'Done' and the results folder, then lists the three main output files (the PDF, REPORT.md, findings.json) so you can spot them among the raw tool files.

4. What happens when you run it (start to finish)

Example: `./audit_repo.sh https://github.com/juice-shop/juice-shop ./report`

#	What happens	Detail
1	**It reads your two inputs**	The web address becomes TARGET, ./report becomes the results folder (lines 29-32). It sees a web address, so mode = repo (line 45).
2	**It downloads a copy**	git clone into ./report/repo, history included (lines 55-58).
3	**It looks around**	Counts files and people, lists key files, finds the most-edited files (lines 68-86).
4	**It writes temporary ingredient lists**	Only where the project has none, without running any of its code (lines 88-100).
5	**The specialists inspect it**	OSV-Scanner, npm audit, Trivy, Gitleaks and Semgrep run one after another; each result goes on the attendance sheet (lines 102-157).
6	**It cleans up**	Temporary lockfiles are deleted (line 169).
7	**The report writer takes over**	Translates, merges, de-duplicates, sorts and numbers all findings (lines 179-375).
8	**You get your files**	Security_Audit_Report.pdf, REPORT.md and findings.json, plus the raw tool outputs, in ./report (lines 377-491).

Is it safe to run? By default the script does **not** run the project's own code (it uses --ignore-scripts, and the build step is off unless you set RUN_BUILD=1). Secret values are hidden in the output files. It only downloads and reads. Still, only scan projects you are allowed to examine, and use a container or virtual machine if you ever switch RUN_BUILD on.

5. Does it work everywhere? Environment by environment

The script needs three things: (1) the ability to **run commands**, (2) the five tools **installed**, and (3) **internet access** to the places the tools talk to (GitHub, the package registries, Semgrep's rulebook, OSV's recall list, Trivy's database). Each environment below differs on those three points.

Where you run it	Will it work?	What to do first	What to watch out for
Your own computer (Mac, Linux; Windows via WSL)	**Yes, best option** No AI limits on what runs	Install the 5 tools once (see section 6).	Nothing special. Full internet, all tools work. Use Docker or a VM if you ever turn on RUN_BUILD.
Claude (chat with code execution and file creation on)	**Yes, mostly** Tested by me: repo and snippet mode both ran	Upload the script and ask Claude to install the tools (prompt in section 6). In my session Claude could install from PyPI, npm and GitHub.	In my session, three tools were blocked from the internet (Semgrep rules, Trivy database, OSV lookup) and showed 'failed'. Gitleaks and npm audit worked. Commands also have a time limit, so ask for a background run.
ChatGPT, normal Chat	**Unlikely** Based on public reports, not tested by me	Cannot reliably install the tools.	Reports say installs and website access are blocked by its container proxy, so most tools would show 'skipped' or 'failed'.
ChatGPT Work (cloud, paid plans)	**Probably** Based on public reports, not tested by me	Same steps as Claude: install the tools, then run.	Reported to have a code environment with internet access that can clone repos and install packages. Needs a paid plan.
GitHub Copilot Chat and code review	**No**	Nothing to do.	It can read the script but cannot run it.
Copilot Agent mode (VS Code) or **Copilot CLI**	**Yes** Runs on your own computer	Install the 5 tools locally first.	It asks your permission before running commands. Stay on 'ask first', do not use fully automatic mode.
Copilot coding agent (cloud, inside GitHub)	**Possible with set-up** Based on public documentation	Preinstall the tools through a copilot-setup-steps.yml file in the repo.	It runs behind a firewall. Allow the tools' servers, or some scanners will fail.

Information about ChatGPT and Copilot comes from public reports and documentation read in October 2026. Features change often, so check their current documentation before relying on it.

How to know what actually worked. Open the **Tool status** table (page 1 of the generated PDF). 'ok' = it ran. 'failed' = it tried and could not (usually no internet). 'skipped' = not installed. 'n/a' = does not apply. If anything says failed or skipped, the findings are a **lower bound**: the real number of problems may be higher.

6. Ready-to-use instructions

On your own computer

```
# 1. Install the tools once
brew install gitleaks trivy osv-scanner      # Mac (Linux: see each tool's website)
pip install semgrep reportlab                # use a virtual environment if pip complains

# 2. Make the script runnable, then run it
chmod +x audit_repo.sh
./audit_repo.sh https://github.com/juice-shop/juice-shop ./report

# 3. Open ./report/Security_Audit_Report.pdf
```

Prompt for Claude, or ChatGPT Work (upload audit_repo.sh first)

```
I have uploaded audit_repo.sh. Please:
1. Install gitleaks, trivy, osv-scanner, semgrep and reportlab
   (use a virtual environment for pip if needed).
2. Run:  bash audit_repo.sh <PASTE REPO URL HERE> ./report
   If it takes longer than your command time limit, run it in the
   background and check the log until it finishes.
3. Give me Security_Audit_Report.pdf.
4. Tell me which tools failed or were skipped, and why.
```

Prompt for Copilot Agent mode or Copilot CLI

```
Run ./audit_repo.sh <PASTE REPO URL HERE> ./report in the terminal.
If a tool is missing, tell me the install command for my system and
wait for me. When it finishes, summarize the Critical and High
findings from ./report/REPORT.md in plain language.
```

To check a single piece of code instead of a whole project

```
./audit_repo.sh ./snippet.py ./report

(Or tell the AI: 'Run audit_repo.sh on the attached file snippet.py'.)
Only the key hunter, code reader and Trivy apply; dependency checks show n/a.
```

Best next step: have an AI double-check the findings

```
Read ./report/findings.json. For every Critical and High finding:
open the file at the given location, tell me whether it is a real
problem or a false alarm, explain why in plain language, and show
the corrected code. Do not change any files yet.
```

Handy switches

Type this	What it does
`RUN_BUILD=1 ./audit_repo.sh ...`	Also installs and tests the project (runs its code; use Docker or a VM).
`REPORT_ONLY=1 ./audit_repo.sh x ./report`	Rebuilds the PDF from results already in ./report, without scanning again. The 'x' is a placeholder.
`NO_PDF=1 ./audit_repo.sh ...`	Skips the PDF. You still get REPORT.md and findings.json.

7. When something looks wrong

What you see	What it means	What to do
`python3 is required`	Python is not installed.	Install Python 3, then run again.
`Target not found: ...`	The path or address does not match a file, folder or web address.	Check for typos; web addresses must start with https://.
`Clone failed`	The project could not be downloaded: wrong address, private repo, or no internet.	Check the address, log in to GitHub, or download it yourself and pass the folder.
`[skipped] gitleaks - not installed`	That tool is missing.	Install it (see section 6) and re-run.
`[failed] osv-scanner ... NOT clean`	It could not reach its online database, or found no ingredient lists.	Check internet or firewall; allow api.osv.dev. Do not treat the missing results as 'no problems'.
`[failed] trivy ... FATAL ... DB`	Trivy could not download its vulnerability database.	Check internet or firewall, then re-run.
`[failed] semgrep ... needs internet`	Semgrep could not download its rulebook.	Check internet or firewall (it needs semgrep.dev).
`PDF skipped: reportlab not installed`	The add-on that draws PDFs is missing.	pip install reportlab (in a virtual environment if needed), then run again with REPORT_ONLY=1.
The run is very slow or the chat says it timed out	Blocked tools retry several times before giving up. Chat tools also limit how long one command can run.	Ask the AI to run it in the background and check progress, or run it on your own computer.
A yellow warning says counts are a 'lower bound'	At least one tool failed or was skipped.	Fix the failed tools and re-run. Do not read the counts as 'all clear'.
Many findings in test folders	Test code often contains fake passwords on purpose.	The script already merges these into one Low line. Spot-check a few.

> **One sentence to remember:** audit_repo.sh is a careful inspector's assistant, not the final judge. It gathers and ranks the evidence; a person (or a second AI) should confirm the important findings before anything is changed.